

The Pickup and Delivery Problem with Time Windows and Incompatibility Constraints in Cold Chain Transportation

Faheng Deng,^a Hu Qin,^a Jiliu Li,^a Chun Cheng^{b,*}

^aSchool of Management, Huazhong University of Science and Technology, Wuhan 430074, China; ^bInstitute of Supply Chain Analytics, Dongbei University of Finance and Economics, Dalian 116025, China

*Corresponding author

Contact: dengfaheng@hotmail.com (FD); tigerqin1980@gmail.com,  <https://orcid.org/0000-0003-4794-2807> (HQ); lijiliu@hust.edu.cn (JL); chun.cheng@polymtl.ca,  <https://orcid.org/0000-0001-9995-4055> (CC)

Received: June 24, 2021

Revised: December 28, 2021; April 28, 2022;
July 5, 2022

Accepted: July 8, 2022

Published Online in Articles in Advance:
August 17, 2022

<https://doi.org/10.1287/trsc.2022.1167>

Copyright: © 2022 INFORMS

Abstract. This study investigates a new variant of the pickup and delivery problem with time windows (PDPTW) applied in cold chain transportation, which quantifies the effect of time on the quality of perishable products. Multiple commodities with incompatibility constraints are considered, where some types of products cannot be transported in a vehicle simultaneously because of their different properties and requirements for storage temperatures. The aim is to determine vehicles' pickup and delivery routes as well as their departure times from the depot such that the travel cost and refrigeration cost of vehicles and the quality decay cost of products are minimized. We formulate this problem as a set partitioning model, which is solved exactly by a tailored branch-and-price (B&P) algorithm. To tackle the asymmetry issue arising from the pricing problem of the B&P framework, we develop a novel asymmetric bidirectional labeling algorithm. Benchmark instance sets based on real-world statistical data and classic PDPTW instance sets are first generated for this problem. Numerical results show that our B&P algorithm can solve most instances to optimality in an acceptable time frame. Moreover, our results demonstrate that integrating the refrigeration and quality decay costs into the objective function can significantly lower the total cost of cold chain transportation activities, compared with the widely adopted objective function minimizing only the travel cost.

Funding: This work was supported by the National Key R&D Program of China [Grant 2018YFB1700600] and the National Natural Science Foundation of China [Grants 72101049, 71971090, 71821001].

Supplemental Material: The online supplement is available at <https://doi.org/10.1287/trsc.2022.1167>.

Keywords: cold chain transportation • perishable product • pickup and delivery • incompatibility constraints • branch-and-price

1. Introduction

Cold chain transportation (CCT) refers to the transportation process of temperature-sensitive products like fruits and vegetables along the supply chain, which plays a significant role in cold chain logistics. In China, we have witnessed the growing popularity of CCT over the past few years, for example, the fresh food logistics of JD.com and Hema Fresh of Alibaba, which is mainly implemented in two modes. One is the trunk transportation with a relatively long time horizon like three days or more, and the other is the urban distribution that often needs to be completed within a day. Unlike normal-temperature transportation, CCT involves more complex cost components; that is, besides the travel cost, we also need to consider the refrigeration cost resulting from using refrigerated vehicles. In addition to supporting vehicle movement, the energy of a refrigerated vehicle is also consumed by the cooling machine to maintain an appropriate temperature range for carried products. Refrigeration costs could be very high because the continuous operation of the cooling machine requires lasting

power inputs until the transportation process is over. Data from vehicle manufacturers show that the fuel consumption per 100 kilometers of a medium-size refrigerated vehicle is up to one-and-a-half times that of its normal-temperature counterpart. For an electric refrigerated vehicle, the recharge mileage with the cooling machine on is also significantly shorter than with the cooling machine off.

In CCT, besides vehicle-related costs, the quality decay cost of perishable products is also a key factor. In China, the cost associated with the quality loss of fruits and vegetables is up to 100 billion RMB per year (Chai 2016). In long-distance trunk transportation, vegetables and fruits are liable to decay because of delays, causing economic losses to suppliers. For example, one type of vegetables with a purchase price of 4 RMB/kg can be sold at 5 RMB/kg when fresh. However, once their freshness reduces and staleness is discernible, they may only be sold at 4.2 RMB/kg. As time goes on, their values continue to decline and may become lower than the purchase price. Public

data show that the deterioration rate of perishable products is about 10% or more, and some even reach 30% during the transportation process (Cnwest 2015). The qualities of perishable commodities affect not only suppliers' costs but also customers' satisfaction, especially those who shop online. Thus, effective operations of CCT are significant to the success of fresh food e-commerce.

Because of the rapid expansion of e-commerce and the improved consumption level, multiproduct transportation is another challenge faced by cold chain logistics companies. First, it is difficult to guarantee commodity qualities while simultaneously transporting multiple products with different optimal storage temperatures (Stellingwerf et al. 2021). For example, at -20°C , the nutrition value of vegetables will be damaged, whereas pork can be preserved well. Table 1 summarizes the temperature requirements of different products in cold chain logistics. We can observe that different temperature ranges are associated with different commodities, and thus refrigerated vehicles with different temperatures are required. Second, incompatibilities could arise because of different product properties; for example, food and chemical products cannot usually be transported together. Therefore, incompatibility constraints should also be taken into account when performing CCT activities. Note that not all products are perishable in a cold chain system; for example, frozen beef can be preserved well for around three months. Thus, no quality decay cost is associated with stable products as their shelf lives far exceed the time horizon, which indicates that we also need to consider the pickup/delivery priority of products based on their shelf lives in order to minimize the total cost of a CCT system.

1.1. Our Contributions

This paper studies a pickup and delivery problem with time windows and incompatibility constraints (PDPTW-IC) in CCT, which is motivated by the practical transportation activities of a Chinese third-party logistics (3PL) company. This company runs refrigerated vehicles to serve transportation requests during a time horizon. Each request includes an origin-destination pair, where a given type of product is first picked up from the origin and then delivered to the destination. Each request is associated with one type of product, and there are

multiple requests with different (perishable or stable) products. Requests with perishable products should be served as soon as possible to control the quality decay cost. However, requests with stable products are only required to be served within the time windows. We determine vehicles' pickup and delivery routes and their departure times from the depot to minimize the total cost. We consider our work makes the following contributions to the literature:

1. We introduce a new variant of the pickup and delivery problem with time windows (PDPTW), that is, PDPTW-IC, where we explicitly consider the incompatibility constraints among multiple commodities, which commonly exist in real-world CCT activities, whereas are often ignored in prior studies. Based on the characteristics of CCT, we propose a comprehensive objective function, including the travel cost, the refrigeration cost, and the quality decay cost.

2. We construct a set partitioning model for PDPTW-IC, which is solved by a tailored branch-and-price (B&P) algorithm. To tackle the asymmetry issue arising from the pricing problem of the B&P framework, we develop a novel asymmetric bidirectional labeling algorithm and prove the corresponding dominance rules. To the best of our knowledge, this paper develops the first exact algorithm for solving routing problems in cold chain logistics.

3. We generate benchmark instance sets based on real-world statistical data and known instance sets in the literature, which will be available to the research community and thus motivate more algorithmic contributions for this type of problem.

4. We conduct extensive numerical tests to evaluate the proposed B&P algorithm and the new objective function. Results show that our algorithm can solve most instances efficiently. Moreover, the new objective function can significantly lower the total cost, compared with the widely adopted objective function minimizing only the travel cost.

The rest of this paper is structured as follows. Section 2 reviews related literature. Section 3 describes the problem and constructs the mathematical model. Section 4 introduces the bidirectional labeling algorithm for the pricing problem. Section 5 presents the branching strategy. Section 6 reports computational results, which is followed by conclusions in Section 7.

Table 1. The Temperature Requirements of Different Products in Cold Chain Logistics

Temperature requirement	Product category
$0^{\circ}\text{C} \sim 4^{\circ}\text{C}$	Chilled poultry meat, aquatic product, nondairy cream cake, juice, and yogurt
$0^{\circ}\text{C} \sim 7^{\circ}\text{C}$	Vegetable, fruit, pasteurized milk, refrigerated egg, egg liquid, and cooked meat
$\leq -18^{\circ}\text{C}$	Frozen poultry meat, frozen aquatic product, frozen beverage, and quick-frozen vegetable
$-23^{\circ}\text{C} \sim -25^{\circ}\text{C}$	Ice cream and popsicle
$\leq -50^{\circ}\text{C}$	Tuna fish

Source. Shanghai Municipal Quality and Technical Supervision Bureau (2007).

2. Literature Review

This section reviews related studies on PDPTW and CCT. A summary of papers on CCT is presented in Table 2.

The PDPTW has many applications, for example, door-to-door passenger transportation; one-to-one freight transportation; and urban courier services, which can be formulated as a two-index model (Ropke, Cordeau, and Laporte 2007; Furtado, Munari, and Morabito 2017), a three-index model (Cordeau 2006, Ropke and Cordeau 2009), and a set partitioning model (Ropke and Cordeau 2009; Baldacci, Bartolini, and Mingozzi 2011). Several exact methods have been proposed to solve the PDPTW model, including the branch-and-cut algorithm (Cordeau 2006; Ropke, Cordeau, and Laporte 2007; Factorovich, Méndez-Díaz, and Zabala 2020); the branch-price-and-cut (BPC) algorithm (Ropke and Cordeau 2009; Cherkesly, Desaulniers, and Laporte 2015; Veenstra et al. 2017; Bettinelli et al. 2019); and set partitioning formulation-based algorithms (Baldacci, Bartolini, and Mingozzi 2011). Costa, Contardo, and Desaulniers (2019) provide a broad overview of BPC algorithms applied to vehicle routing problems (VRPs), where a subsection reviews related contributions on the PDPTW. Pessoa et al. (2020) propose a BPC solver incorporating the best techniques in the literature for generic VRPs. To the best of our knowledge, the authors have developed the state-of-the-art exact algorithm for the PDPTW. Heuristic methods are also developed, for example, the adaptive large neighborhood search (Pisinger and Ropke 2007); the simulated annealing algorithm (Li and Lim 2008, Wang et al. 2015); the hybrid discrete particle swarm optimization method (Goksal, Karaoglan, and Altıparmak 2013); and the tabu search algorithm (Goeke 2019). For more details of PDPTW and its variants, see the review paper by Cordeau, Laporte, and Ropke (2008); Parragh, Doerner, and Hartl (2008a, b); and Qin et al. (2021).

Incompatibility constraints are common in real-world transportation problems. For example, foods and chemical products should be transported separately, and hazardous materials might dangerously react if presented in the same vehicle. To tackle the incompatibility issue, companies can consider utilizing heterogeneous dedicated vehicles to serve conflictive requests. However, this may require an enormous fleet size, especially when the number of requests is large, leading to a waste of resources. In the literature, some authors have incorporated incompatibility constraints into VRPs. For example, Manerba and Mansini (2015) consider the multivehicle traveling purchaser problem with pairwise incompatibility constraints (MVTTPP-PIC), where multiple pairs of products cannot be transported on a vehicle simultaneously. A branch-and-cut algorithm is developed to solve the problem. Gendreau, Manerba, and Mansini (2016) study a special case of MVTTPP-PIC, where the demand for each product is unitary. The

Table 2. A Summary of Cold Chain Transportation Related Works

Authors	Problem	Component of objective function	Commodity	Incompatibility constraints?	Approach for quality decay	Departure time as decision variable?	Solution method
Hsu et al. (2007)	VRPTW	Fixed cost, transportation cost, inventory cost, energy cost, penalty cost	Single	Not applicable	Randomly decrease over time	Yes	Heuristic
Osvold and Stim (2008)	VRPTW	Travel distance, travel time, delay cost, quality decay cost	Single	Not applicable	Linear decay over time	No	Heuristic
Qi and Hu (2020)	VRPTW	Fuel cost, refrigeration cost, cargo damage cost	Single	Not applicable	Linear decay over time	No	Heuristic
Sellingwerf et al. (2021)	VRP	Product decay, CO ₂ emissions, transportation cost, maximum decay	Single	Not applicable	Time- and temperature-dependent kinetic model	No	Commercial solver
Chen et al. (2009)	Production scheduling and VRPTW	Total profit	Multiple	No	Product-specific decay rate	No	Heuristic
Amorim and Almada-Lobo (2014)	VRPTW	Distribution cost, freshness of product	Multiple	No	Linear decay over time	No	Heuristic
Song and Ko (2016)	VRP	Customer satisfaction	Multiple	No	Product-specific decay rate	No	Heuristic
This paper	PDPTW	Travel cost, refrigeration cost, quality decay cost	Multiple	Yes	Linear decay over time	Yes	Exact

authors develop a column generation (CG) approach for the problem. Factorovich, Méndez-Díaz, and Zabala (2020) present a one-to-one pickup and delivery problem with only a single vehicle. In this case, the authors add an extra incompatibility constraint to impose that incompatible pairs of requests cannot be served by the vehicle simultaneously. Ceselli, Righini, and Salani (2009) use a CG algorithm to solve the rich VRP with incompatibility constraints. Battarra, Monaci, and Roberti (2009) consider a multitrip VRP with incompatibility constraints. They minimize the number of used vehicles and develop an iterative algorithm.

In recent years, cold chain transportation of perishable goods has received increasing attention in the context of VRPs. Focusing on a single type of perishable product, Hsu, Hung, and Li (2007) extend the vehicle routing problem with time windows (VRPTW) by considering stochastic travel times affecting the spoilage rate of products. Osvald and Stirn (2008) consider the distribution of fresh vegetables, where perishability is described by a linear model. The problem is modeled as a VRPTW with time-dependent travel times, which is solved by a tabu search-based heuristic approach. Based on the minimum loss of perishable goods, Qi and Hu (2020) study a VRP in emergency cold chain logistics, where fuel costs, refrigeration costs, and cargo damage costs are considered. The authors assume that refrigeration costs and cargo damage costs increase linearly with time. A hybrid heuristic algorithm is developed. Stellingwerf et al. (2021) focus on a quality-driven VRP in fresh food transportation. They propose a time- and temperature-dependent kinetic model to describe the quality decay realistically, where the temperature is an essential factor affecting product quality. The authors consider four types of objectives (i.e., minimizing product decay, carbon dioxide (CO₂) emissions, cost, and maximum decay) and compare their impacts on product quality.

In another line of research, some authors have studied multiproduct distribution in cold chain logistics. For example, Chen, Hsueh, and Chang (2009) unify the production scheduling problem and the VRPTW in a single framework, where suppliers manufacture perishable products and sell to retailers with stochastic demands. Products are assumed to deteriorate at constant rates once manufactured. Different products have different deterioration rates, so those with lower decay rates are scheduled to be manufactured earlier. Product prices depend on the time differences between manufacturing and delivery. The authors formulate the problem as a nonlinear programming model, where the production scheduling part is solved by a constrained Nelder–Mead method, and the distribution part is solved by a heuristic algorithm. Amorim and Almada-Lobo (2014) construct a multiobjective VRP model to measure the relationship between distribution plans and cost-freshness trade-offs. The authors

assume that the shelf lives of perishable products decrease linearly with time. They develop an ϵ -constraint method and a hybrid evolutionary approach to solve small-size and large-size instances, respectively. Song and Ko (2016) address a VRP for food delivery using both refrigerated and general-type vehicles. The objective is to maximize customer satisfaction, which depends on the freshness of delivered products. When starting a trip, the freshness of each product is assumed to be perfect, whereas it reduces over elapsed travel times. The products delivered by refrigerated vehicles have higher levels of freshness than those delivered by general-type vehicles. To solve the problem, the authors derive a priority-based heuristic algorithm. Numerical results confirm that refrigerated vehicles have better performance in perishable product delivery compared with general-type vehicles.

Based on reviewed papers and Table 2, we find that (1) although multiproduct delivery in CCT is studied by some authors, the incompatibility constraints among products are often ignored, whereas these constraints commonly exist in real-world CCT activities; (2) heuristic algorithms are often used to solve routing problems in CCT. To the best of our knowledge, no exact algorithm has been developed.

3. Problem Description and Mathematical Formulation

In this section, we describe the problem and introduce the quality decay function. We then construct a set partitioning model for PDPTW-IC.

3.1. Problem Description

Our problem is defined on a complete directed graph $G = (V, A)$, where $V = \{0, \dots, 2n + 1\}$ is the set of nodes. Nodes 0 and $2n + 1$ denote the starting and ending depots, whereas subsets $V_p = \{1, \dots, n\}$ and $V_d = \{n + 1, \dots, 2n\}$ denote pickup and delivery nodes, respectively. Each request i is associated with a pickup node i and a delivery node $i + n$. Set $A = \{(i, j) : \forall i \in V \setminus \{2n + 1\}, j \in V \setminus \{0\}, i \neq j\}$ are the arcs. A fleet of $|K|$ identical refrigerated vehicles with capacity Q is available. We define customer set as $V_c = V_p \cup V_d$. Each customer has a demand q_i ($q_i > 0, \forall i \in V_p$ and $q_i = -q_{i-n}, \forall i \in V_d$) and must be visited exactly once. The demands at depots are $q_0 = q_{2n+1} = 0$. Each node $i \in V$ is associated with a hard time window $[e_i, l_i]$, where e_i and l_i are the earliest and latest times to start the service at node i , respectively. We let $e_0 = e_{2n+1} = 0, l_0 = l_{2n+1} = T$, where T is the time horizon. Each arc $(i, j) \in A$ is associated with a travel distance d_{ij} and a travel time σ_{ij} , where the latter also includes the service time at node i . The service times at the depots are set to zero. We assume that travel times satisfy the triangle inequality and that travel cost and refrigeration cost are proportional to travel distance and time, respectively. Let ρ_0 be the travel cost per unit distance and ρ_1

be the refrigeration cost per unit time. For each request, only one type of commodity is involved; but this type of commodity may be included in multiple requests. Let φ_i be the commodity type included in request i .

Different from traditional PDPTWs, our problem requires that commodities transported on a vehicle must be compatible at any time of the transportation process—traveling on the roads or waiting at customers. We introduce operators $\oplus$ and $\ominus$ to respectively denote compatibility and incompatibility between two commodities or a commodity and a set. For example, $a \oplus b$ represents that a and b are compatible. Besides vehicle capacity, time window, and incompatibility constraints, a feasible route r also needs to satisfy the pairing and precedence constraint, that is, route r can visit the delivery node $i + n$ only after visiting the pickup node $i \in V_p$. We can then remove arcs $(i + n, i)$, $\forall i \in V_p$ from set A .

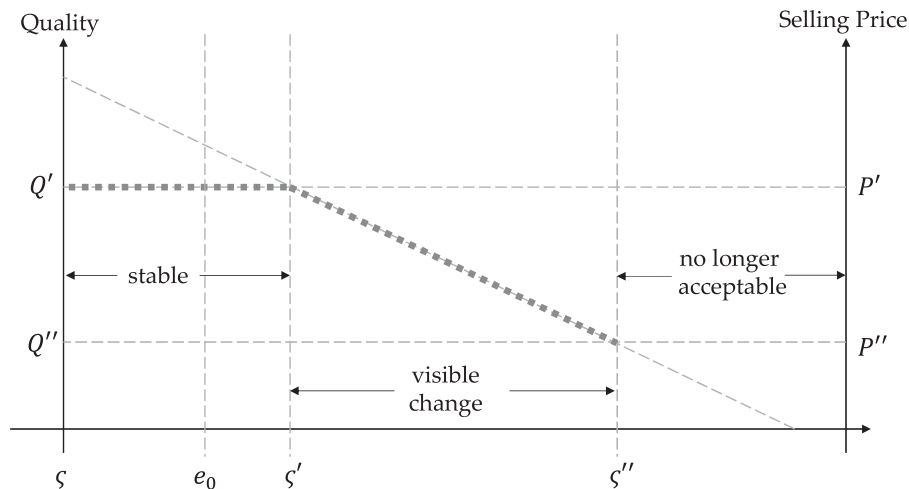
3.2. Quantification of Quality Decay

In cold chain logistics, although there are many approaches to measure the quality decay, most are too complicated and require a large number of input parameters, which are often unavailable in the transportation process. In this study, we extend the linear model proposed by Osvald and Stirn (2008) to estimate the quality decay, as their model requires few parameters. Moreover, the authors indicate that a loss of 20% of quality can be related to a transportation process, denoting three cases: (1) 20% of transported quantity is completely decayed, and 80% is in perfect conditions; (2) the whole transported quantity is evenly decayed so that a product could be sold only at 80% of its original price; or (3) a combination of the first two cases. For completely decayed products, they are often disposed of without any return; thus, case (1) is equivalent to selling all quantities at 80% of the

original prices. Based on these backgrounds, we assume that the quality decay of perishable products leads to a decrease in their market price. This price decrease depends on four parameters: ζ' , ζ'' , P' , and P'' , as represented by Figure 1, where the limited life span of each perishable product is divided into three stages. At the moment of harvest (point ζ), perishable products are in perfect condition. From time ζ to time ζ' , although the quality is reducing, there are no discernible changes; thus, a product can be sold at a good price P' at this stage. The quality decay continues and becomes visible in the second stage, causing the selling price to drop linearly with time. At time ζ'' , the product can only be sold at a very low price P'' . After time ζ'' , the product becomes unacceptable and will be disposed of without any return. We consider the time duration $\zeta' - \zeta$ as the *fresh period* of products because there are no discernible changes. From time ζ to time e_0 , harvested products are collected and taken to pick up customers; and we consider this time duration as the *harvest period*. When everything is ready, our pickup and delivery process begins at e_0 . Note that if the *harvest period* is shorter than the *fresh period*, products have no discernible changes from e_0 to ζ' .

Although cold chain logistics has a very high requirement of transportation timeliness, in real-world applications, perishable products are seldom delivered to customers before time ζ' . And it is a common phenomenon that products decay visibly before delivery. Moreover, once a product becomes unacceptable, it might be disposed of directly and will not be delivered to customers anymore. Thus, we assume that the pickup and delivery process ends before time ζ'' . In addition, as mentioned before, not all cold chain products experience quality loss, that is, the qualities of some products are stable under refrigerated conditions. Thus, no quality decay cost is associated with

Figure 1. A Quality Decay Model Extended from Osvald and Stirn (2008)



stable products. We define $\mathcal{P}$ as the set of all product types, among which some types of products are perishable, denoted as set $\overline{\mathcal{P}}$. Then the rest products in set $\mathcal{P} \setminus \overline{\mathcal{P}}$ are stable. Let t_i be the service starting time (SST) at node i and t_0 be the departure time of a vehicle from depot 0. The quality decay cost $\Lambda(i)$ of request i is calculated as

$$\Lambda(i) = q_i \frac{\max\{t_{i+n} - \zeta'_{\varphi_i}, 0\}}{\zeta''_{\varphi_i} - \zeta'_{\varphi_i}} (P'_{\varphi_i} - P''_{\varphi_i}) \quad \forall \varphi_i \in \overline{\mathcal{P}}, i \in V_p, \quad (1)$$

where $\zeta''_{\varphi_i} - \zeta'_{\varphi_i}$ represents the time span of visible changes of commodity φ_i ; and P'_{φ_i} and P''_{φ_i} are the commodity's market prices at times ζ'_{φ_i} and ζ''_{φ_i} , respectively. If $\varphi_i \in \mathcal{P} \setminus \overline{\mathcal{P}}$, then no decay cost is associated with request i . For notational simplicity, we let

$$\pi_{i+n} = q_i \frac{P'_{\varphi_i} - P''_{\varphi_i}}{\zeta''_{\varphi_i} - \zeta'_{\varphi_i}}, \quad \forall \varphi_i \in \overline{\mathcal{P}}, i \in V_p. \quad (2)$$

If $\varphi_i \in \mathcal{P} \setminus \overline{\mathcal{P}}$, we set $\pi_{i+n} = 0$.

Figure 2 illustrates a feasible solution to the PDPTW-IC, where $\varphi_3, \varphi_4 \in \overline{\mathcal{P}}$ and $\varphi_1, \varphi_2 \in \mathcal{P} \setminus \overline{\mathcal{P}}$. Meanwhile, φ_1 and φ_2 are incompatible. For each customer, the time-related parameters are given in the square brackets, where the first two numbers denote the time window and the third number is the SST at that customer. For the ending depot 9, the last two numbers in the square bracket respectively denote the returning times of vehicles 1 and 2. Other parameters are omitted here because the main purpose of this example is to demonstrate the special features of our problem. As $\varphi_1 \ominus \varphi_2$, vehicle 1 can serve request 2 only after finishing request 1. Because no quality decay cost is associated with these two requests, vehicle 1 should leave depot 0 as late as possible to reduce the waiting times at customers and

thus minimize the refrigeration cost. In contrast, as $\varphi_3 \oplus \varphi_4$, vehicle 2 can first visit all the pickup customers and then the delivery customers as long as other side constraints can be satisfied. Because φ_3 and φ_4 are perishable, vehicle 2 should leave depot 0 as early as possible to control the quality decay cost; however, to minimize the refrigeration cost, vehicle 2 is expected to leave the depot as late as possible. To balance these two types of cost and minimize their sum, the optimal departure time for vehicle 2 is 8:43 in this example. We will show how to deal with these two conflicting objective components in Section 4.

3.3. A Set Partitioning Model

The PDPTW-IC consists of designing vehicle routes and determining the departure time of each route from the depot to serve customers, such that the travel cost, refrigerated cost, and quality decay cost are minimized. To solve PDPTW-IC instances efficiently, this section introduces a set partitioning model, based on which a B&P algorithm is developed in the next section.

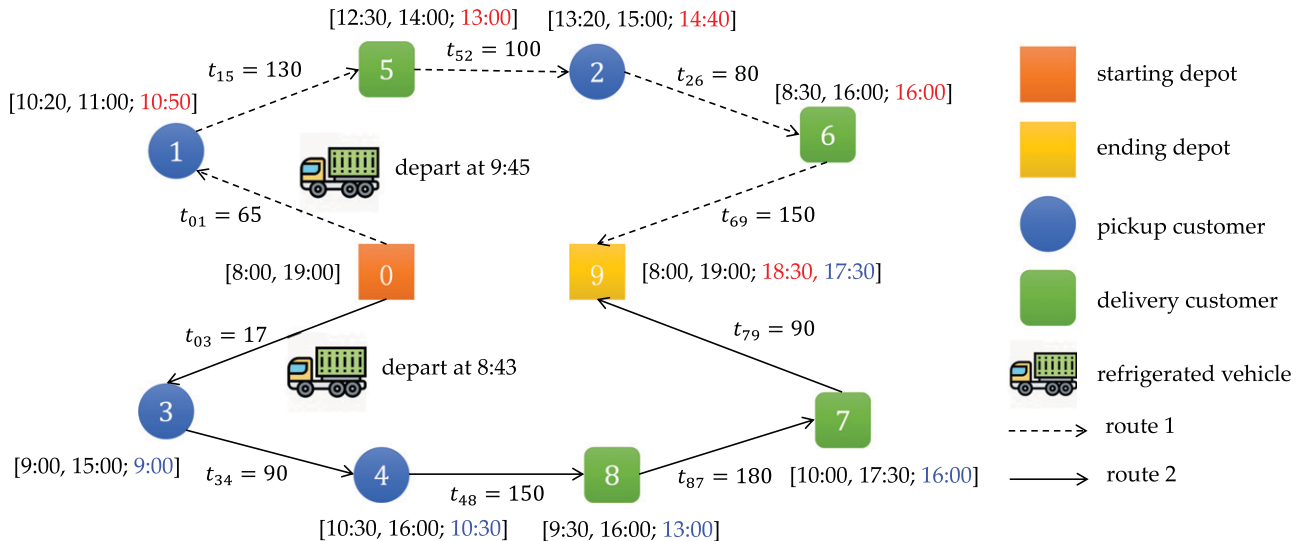
Let $\mathcal{R}$ be the set of all feasible routes and c_r be the cost of route $r \in \mathcal{R}$. For each $r \in \mathcal{R}$, we denote d_r as its total travel distance and assume the last customer in this route is j . Then c_r can be computed as

$$c_r = \rho_0 d_r + \rho_1 (t_j - t_0) + \sum_{i \in \Delta_r} \Lambda(i), \quad (3)$$

where Δ_r is the set of pickup customers in route r and the three terms represent the travel cost, the refrigeration cost, and the quality decay cost, respectively.

A binary parameter a_{ir} equals one if request $i \in V_p$ is served by route r and zero otherwise. Let θ_r be a binary variable equaling one if route $r \in \mathcal{R}$ is selected

Figure 2. (Color online) Illustration of a Feasible Solution to the PDPTW-IC



and zero otherwise. The set partitioning model is then constructed as

$$\min \sum_{r \in \mathcal{R}} c_r \theta_r \quad (4)$$

$$\text{subject to } \sum_{r \in \mathcal{R}} a_{ir} \theta_r = 1 \quad \forall i \in V_p, \quad (5)$$

$$\sum_{r \in \mathcal{R}} \theta_r \leq |K|, \quad (6)$$

$$\theta_r \in \{0, 1\} \quad \forall r \in \mathcal{R}. \quad (7)$$

The objective function (4) minimizes the total cost of selected routes. Constraints (5) force that each request is served exactly once. Constraints (6) ensure that at most $|K|$ routes are selected in the final solution. Constraints (7) are integrality constraints.

Because of the large size of set $\mathcal{R}$, it is impractical to enumerate all the feasible routes, even for small-size instances. Thus, we utilize a CG algorithm (Desaulniers, Desrosiers, and Solomon 2005) to solve the linear relaxation problem (LRP) at each node of the branch-and-bound (B&B) tree, leading to a B&P method. In the CG algorithm framework, a restricted master problem (RMP) is obtained by considering the LRP with a reduced subset $\mathcal{R}' \subseteq \mathcal{R}$, which can be efficiently solved to optimality by the simplex method. After solving the RMP, dual solutions are obtained and passed to a pricing problem to check if any route $r' \in \mathcal{R}$ with a negative reduced cost exists. If so, route r' is added to set $\mathcal{R}'$, and the RMP is solved again. Otherwise, the current solution is optimal for the LRP.

We let $\mu_{ir}, i \in V_p$ and μ_0 be the dual variables associated with Constraints (5) and (6), respectively. We set $\mu_{2n+1} = 0$ and $\mu_i = 0, \forall i \in V_d$. Then the reduced cost $\bar{c}_r$ of route $r \in \mathcal{R}$ is

$$\bar{c}_r = c_r - \sum_{i \in V} a_{ir} \mu_i = c_r - \sum_{i \in V_r} \mu_i, \quad (8)$$

where V_r is the set of nodes in route r .

4. The Pricing Problem

The pricing problem of PDPTW-IC is an elementary shortest path problem with resource and incompatibility constraints, which is NP-hard. To efficiently solve it, a tailored bidirectional labeling algorithm is developed, which has been proved to be more efficient than the monodirectional labeling algorithm (Righini and Salani 2006, Gschwind et al. 2018). We generate labels in both forward and backward directions and combine them to form completed routes. We first introduce the functions for calculating the refrigeration and quality decay costs of a route in Section 4.1, which will be used in the bidirectional labeling algorithm. We then present the forward and backward labeling algorithms in Sections 4.2 and 4.3, respectively. The label joining procedure is introduced in Section 4.4. We finally present an *ng*-route relaxation

technique in Section 4.5 to further accelerate the labeling algorithm.

4.1. Forward and Backward Cost Functions

Besides customer visiting sequences, the refrigeration and quality decay costs are also affected by vehicle departure times (from the depot) derived from some functions' limits. To minimize the quality decay cost, vehicles are expected to leave the depot as early as possible; however, to minimize the refrigeration cost, vehicles are expected to start trips as late as possible (to reduce waiting times at customers) as long as time window constraints can be met. These conflicting objective components lead to asymmetric function extensions, which are explained in the following.

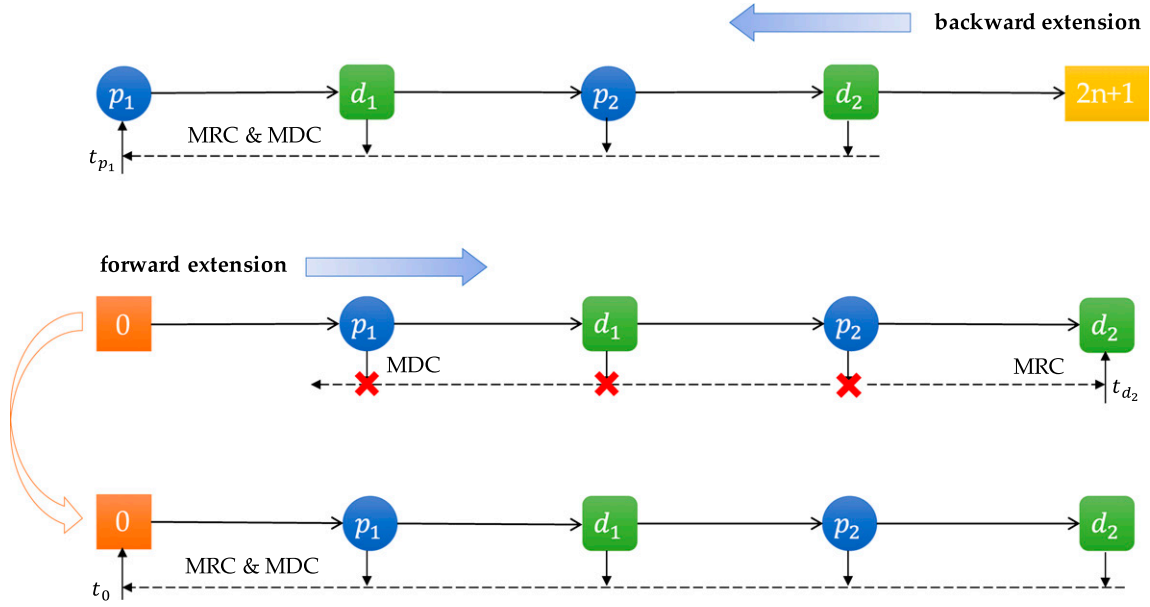
Take Figure 3(a) as an example, and suppose that a backward partial route is extended from node $2n+1$ to node p_1 . Given an SST t_{p_1} at node p_1 , there are many feasible values for the SST t_{d_1} at its successor node d_1 , as long as the time window constraint at d_1 can be satisfied. To minimize the refrigeration cost, t_{d_1} is expected to be close to t_{p_1} , which also holds for the objective minimizing the decay cost. However, this analysis process does not apply to the forward extension. Specifically, suppose that a forward partial route is extended from node 0 to node d_2 , as shown in Figure 3(b). Given an SST t_{d_2} at node d_2 , for its predecessor node p_2 , to minimize the refrigeration cost, the SST t_{p_2} at p_2 is expected to be close to t_{d_2} (then the route duration will be shorter); however, to minimize the quality decay cost, t_{p_2} is expected to be close to e_0 (i.e., pickup and deliver commodities as early as possible). Thus, it is difficult to decide a time t_{p_2} for node p_2 such that the refrigeration cost and quality decay cost are simultaneously minimized. Similar analyses also apply to nodes d_1 and p_1 . Instead, as shown in Figure 3(c), given the SST t_0 at node 0, for any node in the forward partial route, its SST is expected to be close to t_0 , which can simultaneously optimize both cost components. To conclude, the specific characteristics of PDPTW-IC make our label extension methods and B&P algorithm different from those developed in the literature.

4.1.1. Forward Cost Functions. For a forward partial or completed route $r^f = (v_0, v_1, \dots, v_m = i)$ beginning at node $v_0 = 0$ and ending at node i , let α and β be the earliest and latest SST at node i , respectively, such that the time window constraints of customers in route r^f are satisfied. We define three functions of t_0 as follows:

- $\delta(t_0)$: the SST at the ending node i , whose range is restricted to $[\alpha, \beta]$ to ensure the time window feasibility of customers in r^f ;
- $\tau^d(t_0)$: the quality decay cost of route r^f ;
- $\tau^f(t_0)$: the sum of the refrigeration and quality decay costs of route r^f .

Note that $\tau^d(t_0)$ is an auxiliary cost function that helps simplify the derivation process of function $\tau^f(t_0)$. The

Figure 3. (Color online) Asymmetric Extensions



Note. MRC, minimize refrigeration cost; MDC, minimize quality decay cost.

following theorems and corollary indicate that the three functions admit piecewise linear convex expressions.

Theorem 1. $\delta(t_0)$ is a piecewise linear nondecreasing convex function expressed as

$$\delta(t_0) = \max\{\omega + t_0, \alpha\}, \quad (9)$$

where ω is the total travel time of route r^f .

Proof. See the online supplement. $\square$

Theorem 2. $\tau^d(t_0)$ is a piecewise linear nondecreasing convex function.

Proof. See the online supplement. $\square$

Corollary 1. $\tau^f(t_0)$ is a piecewise linear convex function.

Proof. See the online supplement. $\square$

To help understand these three functions, we give their visualizations in Figure 4.

4.1.2. Backward Cost Functions. For a backward partial or completed route $r^b = (v_0, v_1, \dots, v_m = i)$ beginning at node $v_0 = 2n + 1$ and ending at node i , we define two functions of t_i as follows:

- $\tau^b(t_i)$: the sum of the refrigeration and quality decay costs of route r^b ;
- $\tau^c(t_i)$: the cost evaluation function of route r^b , equaling $\tau^b(t_i) + \rho_1 t_i$.

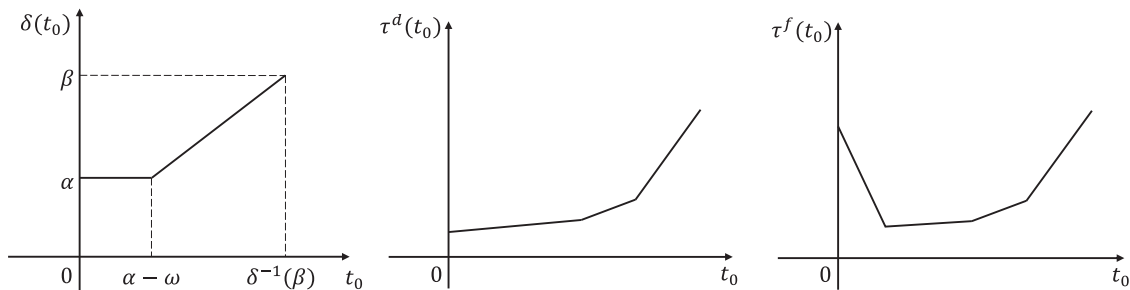
Similar to the forward cost function $\tau^d(t_0)$, $\tau^c(t_i)$ is also an auxiliary function as it is more straightforward to derive it than $\tau^b(t_i)$.

Theorem 3. $\tau^c(t_i)$ is a piecewise linear nondecreasing convex function.

Proof. See the online supplement. $\square$

Based on the definition of $\tau^c(t_i)$, we can obtain $\tau^b(t_i) = \tau^c(t_i) - \rho_1 t_i$. The visualizations of functions $\tau^c(t_i)$ and $\tau^b(t_i)$ can refer to those of $\tau^d(t_0)$ and $\tau^f(t_0)$ as

Figure 4. Visualizations of Functions $\delta(t_0)$, $\tau^d(t_0)$, and $\tau^f(t_0)$



presented in Figure 4, respectively. After having the cost functions, we can rewrite the reduced cost $\bar{c}_r$ of route $r \in \mathcal{R}$ in (8) as

$$\bar{c}_r = \rho_0 d_r + \min_{t_0 \in [\alpha, \beta]} \tau^b(t_0) - \sum_{i \in V_r} \mu_i. \quad (10)$$

The $\min$ term means that we set the departure time from the depot as the one minimizing $\tau^b(t_0)$.

Besides the asymmetric function extensions introduced earlier, we note that there also exists asymmetry in the definitions of forward and backward functions. Specifically, $\tau^d(t_0)$ only accounts for costs paid in the forward part of a route, while $\tau^c(t_i)$ also accounts for the term $\rho_1 t_i$ that is already paid in the forward part of a route.

4.2. Forward Labeling Algorithm

This section first defines the label associated with a forward route and then presents the method for label extension. Finally, we introduce the dominance rules.

4.2.1. Label Definition and Extension. A forward route $r^f = (v_0, v_1, \dots, v_m = i)$ is represented by a label $L = (i, \Gamma, U, C, \omega, \alpha, \beta, \mathcal{D}, \mathcal{F})$, which includes the following information:

- i : the current node of the label;
- Γ : the set of nodes that have not been visited by route r^f ;
- U : the set of unfinished requests, that is, the set of pickup customers that have been served by route r^f , but their delivery customers have not been served;
- C : the difference between the accumulated dual value and the accumulated travel cost;
- ω : the total travel time of route r^f ;
- $[\alpha, \beta]$: the domain of the SST at node i to ensure the time window feasibility of all nodes in r^f ;
- $\mathcal{D} = \{\mathcal{D}_h = (\bar{u}_h, \bar{g}_h)\}_{h=1, \dots, |\mathcal{D}|}$: the parameters used to express function $\tau^d(t_0)$ (see the proof of Theorem 2);
- $\mathcal{F} = \{\mathcal{F}_h = (u_h, g_h)\}_{h=1, \dots, |\mathcal{F}|}$: the parameters used to express function $\tau^f(t_0)$ (see the proof of Corollary 1).

We denote $\Psi(U) = \{\varphi_v \mid v \in U\}$ as the set of commodities on board and $W(U) = \sum_{v \in U} q_v$ as the load on board. Label L can be extended to node $j \in \Gamma$ only if $W(U) + q_j \leq Q$ and $\max\{e_j, \alpha + \sigma_{ij}\} \leq \max\{e_j, \min\{l_j, \beta + \sigma_{ij}\}\}$, ensuring the vehicle capacity and time window constraints are respected. Meanwhile, a legal extension must satisfy one of the following conditions:

$$j \in V_p \wedge (\varphi_j \oplus \Psi(U)), \quad (11)$$

$$j \in V_d \wedge (v_{j-n} \in U), \quad (12)$$

$$j = 2n + 1 \wedge U = \emptyset. \quad (13)$$

Constraint (11) means that if j is a pickup node, then the compatibility constraint must be satisfied. Constraint (12) imposes that a route can visit a delivery node only if the corresponding pickup node has been

visited. Constraint (13) indicates that the vehicle can return to the ending depot only after finishing all assigned requests.

The resources associated with the new label $L' = (j, \Gamma', U', C', \omega', \alpha', \beta', \mathcal{D}', \mathcal{F}')$ extended from L are calculated as follows:

$$\Gamma' = \Gamma \setminus \{j\} \setminus \mathcal{A} \setminus \mathcal{B}, \quad (14)$$

$$C' = C + \mu_j - \rho_0 d_{ij}, \quad (15)$$

$$\omega' = \omega + \sigma_{ij}, \quad (16)$$

$$\alpha' = \max\{e_j, \alpha + \sigma_{ij}\}, \quad (17)$$

$$\beta' = \max\{e_j, \min\{l_j, \beta + \sigma_{ij}\}\}, \quad (18)$$

$$\text{if } j \text{ is the ending depot } 2n + 1: \quad (19)$$

$$\mathcal{F}' = \mathcal{F},$$

$$\text{if } j \text{ is a pickup customer:} \quad (20)$$

$$U' = U \cup \{j\}, \quad (21)$$

$$\mathcal{D}' = \mathcal{D}, \quad (22)$$

$$\mathcal{F}' = \{(\bar{u}'_h - \rho_1, \rho_1 \alpha' + \bar{g}'_h)\}_{\mathcal{D}'_h \in \mathcal{D}'} \cup \{(\bar{u}'_h, \bar{g}'_h + \rho_1 \omega')\}_{\mathcal{D}'_h \in \mathcal{D}'}, \quad (23)$$

$$\text{if } j \text{ is a delivery customer:} \quad (24)$$

$$U' = U \setminus \{j - n\},$$

$$\mathcal{D}' = \{(\bar{u}_h + \pi_j, \pi_j(\omega' - \zeta'_j) + \bar{g}_h)\}_{\mathcal{D}_h \in \mathcal{D}} \cup \{(\bar{u}_h, \bar{g}_h + \pi_j(\alpha' - \zeta'_j))\}_{\mathcal{D}_h \in \mathcal{D}} \cup \mathcal{D}, \quad (25)$$

$$\mathcal{F}' = \{(\bar{u}'_h - \rho_1, \rho_1 \alpha' + \bar{g}'_h)\}_{\mathcal{D}'_h \in \mathcal{D}'} \cup \{(\bar{u}'_h, \bar{g}'_h + \rho_1 \omega')\}_{\mathcal{D}'_h \in \mathcal{D}'}, \quad (26)$$

where $\mathcal{D}'_h = (\bar{u}'_h, \bar{g}'_h)$ is the h^{th} element of set $\mathcal{D}'$. In Equation (14), $\mathcal{A}$ and $\mathcal{B}$ are the sets of unreachable customers from the new label L' . Specifically, traveling directly from node j to $v \in \mathcal{A}$ will violate the time window at customer v , and traveling to $v \in \mathcal{B}$ will violate the combination of the incompatibility constraints and time window constraints. Namely, if $z \in U'$ and $\varphi_z \ominus \varphi_v$, then visiting node v by the route associated with label L' will require the vehicle to deliver commodity φ_z first, that is, visiting node $z + n$ before node v . In this case, we can label node v as unreachable if its time window constraint cannot be satisfied. Mathematically, sets $\mathcal{A}$ and $\mathcal{B}$ are defined as

$$\mathcal{A} = \{v \in V_p \mid \alpha' + \sigma_{jv} > l_v\}, \quad (27)$$

$$\mathcal{B} = \{v \in V_p \mid \exists z \in U' \wedge (\varphi_z \ominus \varphi_v), \max\{\alpha' + \sigma_{j, z+n}, e_{z+n}\} + \sigma_{z+n, v} > l_v\}. \quad (28)$$

When defining these two sets, we do not consider the delivery customer of request v because we can set $l_v := \min\{l_v, l_{v+n} - \sigma_{v, v+n}\}$, $\forall v \in V_p$ to ensure $l_v + \sigma_{v, v+n} \leq l_{v+n}$ (Ropke and Cordeau 2009). This setting will not affect the feasible solutions of the original problem. The derivation of Equation (26) is straightforward; thus, we only detail how Equation (27) is obtained. Specifically, for label L' , we assume customer $v \in V_p$

satisfies the conditions in set $\mathcal{B}$. To prove that v is unreachable for label L' , we consider two cases: (i) Customer $z + n$ is not visited yet, that is, commodity φ_z is still on board. Then we cannot visit customer v because its associated commodity φ_v is incompatible with φ_z . (ii) Customer $z + n$ has already been visited. Let α'' be the earliest SST at customer $z + n$, then we must have $\alpha'' \geq \max\{\alpha' + \sigma_{j,z+n}, e_{z+n}\}$, where the first term of the right-hand side is obtained by letting the vehicle travel directly from customer j to $z + n$ because visiting other intermediate nodes before $z + n$ will increase the travel time due to the triangle inequality. Subsequently, we can obtain $\alpha'' + \sigma_{z+n,v} \geq \max\{\alpha' + \sigma_{j,z+n}, e_{z+n}\} + \sigma_{z+n,v} > l_v$, making customer v unreachable. The idea of defining unreachable sets comes from Feillet et al. (2004). The logic behind it is that when extending to a new label, we remove the nodes that cannot be visited in any extension of the corresponding path, either because they have been visited or because of resource constraints. This idea has the potential for more successful dominance and thus speeding up the labeling process.

After updating sets $\mathcal{D}$ and $\mathcal{F}$, an AERASE procedure detailed in Algorithm 1 is applied to erase redundant lines in these sets. In row 7, the expression $g_i \geq g_j$ denotes the redundant relation between two lines. In row 16, the expression $u_i\zeta + g_i \geq u_k\zeta + g_k$ means that if the intersection point ζ of lines $u_it + g_i$ and $u_jt + g_j$ lies above line $u_kt + g_k$, then line $u_kt + g_k$ is redundant. The redundant relations among lines are illustrated in Figure 5.

Algorithm 1 (The Pseudocode of the AERASE Procedure)

- 1: **Input:** Set A for calculating the forward or backward cost function.
- 2: Sort the element $(u, g) \in A$ by u in nondecreasing order. If u is identical, we sort the elements by g .
- 3: Initialize set $C_L = \emptyset$.
- 4: **for** $i = |A|, \dots, 2$ **do**
- 5: **if** $(u_i, g_i) \notin C_L$ **then**
- 6: **for** $j = i - 1, \dots, 1$ **do**
- 7: **if** $((u_j, g_j) \notin C_L) \wedge (g_i \geq g_j)$ **then**
- 8: $C_L := C_L \cup \{(u_j, g_j)\}$.
- 9: $A := A \setminus C_L$ and $C_L = \emptyset$.
- 10: **for** $i = 1, \dots, |A| - 2$ **do**
- 11: **if** $(u_i, g_i) \notin C_L$ **then**
- 12: **for** $j = |A|, \dots, i + 2$ **do**
- 13: **if** $(u_j, g_j) \notin C_L$ **then**
- 14: **for** $k = j - 1, \dots, i + 1$ **do**
- 15: Let $\zeta = (g_i - g_j) / (u_j - u_i)$.
- 16: **if** $((u_k, g_k) \notin C_L) \wedge (u_i\zeta + g_i \geq u_k\zeta + g_k)$ **then**
- 17: $C_L := C_L \cup \{(u_k, g_k)\}$.
- 18: $A := A \setminus C_L$.
- 19: **Output:** Set A after removing redundant lines.

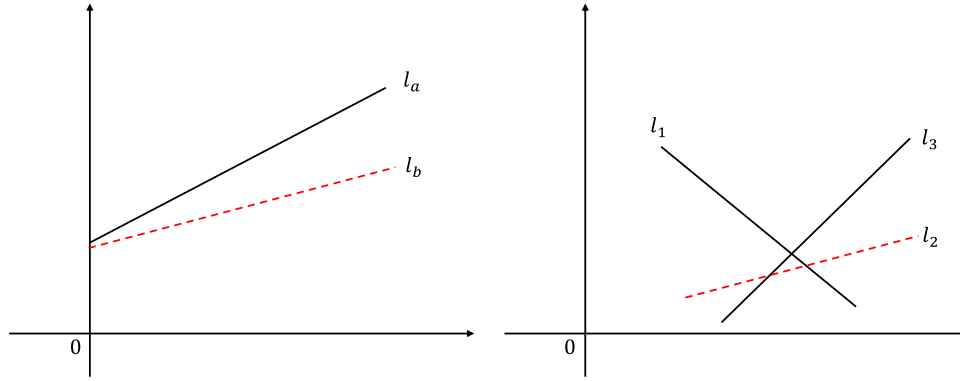
For the forward partial route $r^f = (0)$, because $\tau^d(t_0) = 0$, we can simply set $\mathcal{D} = \{(0, 0)\}$. When conducting the forward extension operation, a special case may arise. That is, the range of function $\delta(t_0)$ reduces to a single value (i.e., $\alpha = \beta$) at some nodes. Let $[\alpha_{v_k}, \beta_{v_k}]$ be the range of the SST at node v_k ($k \in [0, m]$). For two successive nodes i and j of a route where $\alpha_i < \beta_i$, the special case happens when (1) $l_i + \sigma_{ij} \leq e_j$, then the SST at node j can only be e_j regardless of when node i is served; (2) $\alpha_i + \sigma_{ij} = l_j$, then the SST at node j can only be l_j and the range of the SST at node i should be updated to $[\alpha_i, \alpha_i]$. Moreover, we also need to update the ranges of nodes before i , to respect the time window constraints at j and its successors. Note that if the SST at a node is fixed, the time schedules of its successors are also fixed.

4.2.2. Dominance Rules. To accelerate the labeling algorithm, dominance rules are often utilized to eliminate unpromising labels. Before moving forward, we first give some definitions. Let $\mathbb{P}(L)$ be the partial path associated with label L . A feasible partial path that can be used to extend $\mathbb{P}(L)$ is called as a *feasible completion* for label L . Let $\Omega(L)$ be the set of all feasible completions of label L . For a completion $\xi \in \Omega(L)$, we use $\mathbb{P}(L) \otimes \xi$ to denote a feasible completed route obtained by combining $\mathbb{P}(L)$ and ξ , and $\bar{c}(\mathbb{P}(L) \otimes \xi)$ is the resulting reduced cost. For two labels L_1 and L_2 ending at the same node, we use $L_1 \prec L_2$ to indicate that label L_1 dominates label L_2 . The standard dominance rule introduced by Dabia et al. (2013) is presented in Proposition 1.

Proposition 1. For labels L_1 and L_2 ending at the same node, $L_1 \prec L_2$ if

1. $\Omega(L_2) \subseteq \Omega(L_1)$,
2. $\bar{c}(\mathbb{P}(L_2) \otimes \xi) \geq \bar{c}(\mathbb{P}(L_1) \otimes \xi)$, $\forall \xi \in \Omega(L_2)$.

Condition 1 means that L_1 can combine with more feasible completions to form a completed route than L_2 . As the departure time from depot 0 needs to be decided in the forward extension, it is not straightforward to check this condition for our dominance rule. On the other hand, because the SST t_i at the current node i affects the feasibility of the completions due to the time window constraints, the larger the range of t_i , the more completions can be combined with the path of the corresponding label. Because we have recorded the range of t_i , that is, $[\alpha, \beta]$ in the forward extension, one necessary condition to ensure condition 1 is $[\alpha_2, \beta_2] \subseteq [\alpha_1, \beta_1]$. Therefore, to derive the forward dominance rule for our problem, we first use $[\alpha_2, \beta_2] \subseteq [\alpha_1, \beta_1]$ together with other necessary conditions to guarantee condition 1. If we can ensure condition 2

Figure 5. (Color online) Visualizations of Redundant Lines (l_b and l_2) in Sets $\mathcal{D}$ and $\mathcal{F}$ 

holds for any $t_i \in [\alpha_2, \beta_2]$, we are done. To do so, we must calculate the reduced costs of $\mathbb{P}(L_1)$ and $\mathbb{P}(L_2)$. From Equation (8), to compute the reduced cost, we first need to know the total cost of a forward route. However, because the argument of the forward cost functions is t_0 (not t_i), we must derive a function, say $\eta(t_i)$, to compute the forward cost under the condition that t_i (instead of t_0) is given.

For a forward route $r^f = (v_0, v_1, \dots, v_m = i)$, function $\eta(t_i)$ denotes its refrigeration and quality decay costs, which will be used in both the forward dominance rules and the label joining procedure. The logic behind defining $\eta(t_i)$ is that to use $\tau^f(t_0)$ to compute the forward cost, we first need to transform t_i to t_0 via a function. Note that besides $\alpha < \beta$, the definition of $\eta(t_i)$ also applies to the special case $\alpha = \beta$. For brevity purpose, we provide the detailed derivation of $\eta(t_i)$ in the online supplement. Based on the preceding analyses, we derive another dominance rule in Proposition 2, which is more straightforward than Proposition 1 as it compares the accumulated costs and resource consumptions associated with two labels.

Proposition 2 (Forward Dom). For labels L_1 and L_2 ending at the same node, $L_1 \prec L_2$ if

1. $\Gamma_2 \subseteq \Gamma_1$,
2. $U_1 \subseteq U_2$,
3. $[\alpha_2, \beta_2] \subseteq [\alpha_1, \beta_1]$,
4. $\eta_1(t_i) - C_1 \leq \eta_2(t_i) - C_2, \forall t_i \in [\alpha_2, \beta_2]$.

Proof. See the online supplement. $\square$

The dominance rule in Proposition 2 requires the comparison of two complex functions, that is, $\eta_1(t_i)$ and $\eta_2(t_i)$, which is difficult to be implemented. Because the expressions of $\delta(t_0)$ and $\tau^f(t_0)$ are already known, we propose another dominance rule in Proposition 3 that is easier to handle.

Proposition 3 (Forward Dom*). For labels L_1 and L_2 ending at the same node, $L_1 \prec L_2$ if

1. $\Gamma_2 \subseteq \Gamma_1$,
2. $U_1 \subseteq U_2$,
3. $[\alpha_2, \beta_2] \subseteq [\alpha_1, \beta_1]$,

4. $\tau_2^f(t_0) - \bar{\tau}_1^f(t_0) \geq C_2 - C_1, \forall t_0 \in [\alpha_2 - \omega_2, \beta_2 - \omega_2]$ if $\alpha_2 < \beta_2$,

5. $\eta_1(\alpha_2) - C_1 \leq \eta_2(\alpha_2) - C_2$,

where $\bar{\tau}_1^f(t_0) = \tau_1^f(t_0 + \omega_2 - \omega_1)$.

Proof. See the online supplement. $\square$

We choose time as the resource bounding for label extensions and denote Θ^f (Θ^b) as the threshold for stopping forward (backward) extension. The process of generating forward labels is given in Algorithm 2, where $\mathcal{L}$ is the set of labels that have not been extended and set $\bar{\mathcal{L}}_i$ stores labels ending at customer $i \in V_c$ that are not dominated. With a slight abuse of notation, we denote $\bar{c}(L)$ as the reduced cost associated with label L , $\alpha(L)$ as the earliest SST time at the ending node of label L , and $\Gamma(L)$ as the set of nodes that have not been extended by label L . In rows 17 and 20, Proposition 3 is used to prune dominated labels.

Algorithm 2 (Forward Labeling(Θ^f): The Process of Generating Forward Labels)

- 1: **Define** L_k : a label ending at node k ; R_s : the set of labels associated with a completed route.
- 2: **Initialize** $L_0 = (0, V \setminus \{0\}, \emptyset, 0, 0, e_0, l_0, \{(0, 0)\}, \emptyset)$; $\mathcal{L} = \{L_0\}$; $\bar{\mathcal{L}}_i = \emptyset, \forall i \in V_c$.
- 3: **while** $\mathcal{L} \neq \emptyset$ **do**
- 4: $N_c = \emptyset$; select the first element L_k of set $\mathcal{L}$ and update $\mathcal{L} := \mathcal{L} \setminus \{L_k\}$.
- 5: **if** $k = 2n + 1$ **then**
- 6: **if** $\bar{c}(L_k) < 0$ **then**
- 7: $R_s := R_s \cup \{L_k\}$.
- 8: **else if** $\alpha(L_k) < \Theta^f$ **then**
- 9: **for** $j \in \Gamma(L_k)$ **do**
- 10: **if** label L_k can be extended to node j **then**
- 11: Generate label L_j by extending L_k to j and update $\mathcal{L} := \mathcal{L} \cup \{L_j\}$.
- 12: **if** $j \neq 2n + 1$ **then**
- 13: $N_c := N_c \cup \{j\}, \bar{\mathcal{L}}_j := \bar{\mathcal{L}}_j \cup \{L_j\}$.
- 14: **for** $i \in N_c$ **do**
- 15: $D = \emptyset$; select the last element L_i of set $\bar{\mathcal{L}}_i$.
- 16: **for** $L_i \in \bar{\mathcal{L}}_i$ **do**

17: **if** $L_i \prec L_l$ **then**
18: $D := D \cup \{L_l\}$.
19: **Break;**
20: **else if** $L_l \prec L_i$ **then**
21: $D := D \cup \{L_i\}$.
22: $\bar{L}_i := \bar{L}_i \setminus D$.

4.3. Backward Labeling Algorithm

For a backward route $r^b = (v_0, v_1, \dots, v_m = i)$, we define the corresponding label as $L = (i, \Gamma, U, C, \alpha, \beta, C)$. Attributes i , Γ , C , α , and β have the same meanings as their counterparts in the forward label. The definitions of other attributes are as follows:

- U : the set of delivery customers that have been served by route r^b but their pickup customers have not been served;
- $C = \{C_h = (\hat{u}_h, \hat{g}_h)\}_{h=1, \dots, |C|}$: the parameters used to express function $\tau^c(t_i)$ (see the proof of Theorem 3).

We only record $\tau^c(t_i)$ in the labeling process, because we can get $\tau^b(t_i)$ easily once $\tau^c(t_i)$ is known and it is sufficient to only use $\tau^c(t_i)$ in the backward dominance. Label L can be extended to node $j \in \Gamma$ only if $W(U) + q_j \geq -Q$ and $e_j \leq \min\{l_j, \beta - \sigma_{ji}\}$, guaranteeing the vehicle capacity and time window constraints are respected. Moreover, a legal extension should also meet one of the following three constraints:

$$j \in V_d \wedge (\varphi_j \oplus \Psi(U)), \quad (28)$$

$$j \in V_p \wedge (v_{j+n} \in U), \quad (29)$$

$$j = 0 \wedge U = \emptyset, \quad (30)$$

which have similar meanings as their counterparts (11)–(13).

The resources associated with the new label $L' = (j, \Gamma', U', C', \alpha', \beta', C')$ extended from L are calculated as follows:

$$C' = C + \mu_j - \rho_0 d_{ji}, \quad (31)$$

$$\alpha' = e_j, \quad (32)$$

$$\beta' = \min\{l_j, \beta - \sigma_{ji}\}, \quad (33)$$

$$\text{if } j \text{ is a pickup customer or the starting depot } 0: \quad (34)$$

$$U' = U \setminus \{j+n\},$$

$$C' = \{(0, \hat{u}_h e_i + \hat{g}_h)\}_{C_h \in C} \cup \{(\hat{u}_h, \hat{u}_h \sigma_{ji} + \hat{g}_h)\}_{C_h \in C}, \quad (35)$$

$$\text{if } j \text{ is a delivery customer:} \quad (36)$$

$$U' = U \cup \{j\},$$

$$C' = \{(\hat{u}_h + \pi_j, \hat{u}_h \sigma_{ji} + \hat{g}_h - \pi_j \sigma'_{ji})\}_{C_h \in C} \cup \{(\hat{u}_h, \hat{u}_h \sigma_{ji} + \hat{g}_h)\}_{C_h \in C} \cup \{(\pi_j, \hat{u}_h e_i + \hat{g}_h - \pi_j \sigma'_{ji})\}_{C_h \in C} \cup \{(0, \hat{u}_h e_i + \hat{g}_h)\}_{C_h \in C}. \quad (37)$$

The update of Γ' is the same as its forward counterpart (14). Sets $\mathcal{A}$ and $\mathcal{B}$ are defined as

$$\mathcal{A} = \{v \in V_d \mid \beta' - \sigma_{vj} < e_v\}, \quad (38)$$

$$\mathcal{B} = \{v \in V_d \mid \exists z \in U' \wedge (\varphi_z \ominus \varphi_v), \min\{\beta' - \sigma_{z-n,j}, l_{z-n}\} - \sigma_{v,z-n} < e_v\}. \quad (39)$$

We note that the backward extension starts from the first node after depot $2n+1$ and ends at depot 0. The update of set $\mathcal{C}$ can refer to the proof process of Theorem 3, and the AERASE procedure is applied after its update. When $r^b = (v_0, v_1)$, according to $\tau^c(t_{v_1}) = (\rho_1 + \pi_{v_1})t_{v_1}$, we can simply set $\mathcal{C} = \{(\rho_1 + \pi_{v_1}, 0)\}$. The dominance rule used in backward labeling is presented in Proposition 4. The process of generating backward labels is similar to that for producing forward labels as presented in Algorithm 2.

Proposition 4 (Backward Dom). For labels L_1 and L_2 ending at the same node i , $L_1 \prec L_2$ if

1. $\Gamma_2 \subseteq \Gamma_1$,
2. $U_1 = U_2$,
3. $[\alpha_2, \beta_2] \subseteq [\alpha_1, \beta_1]$,
4. $\tau_1^c(t_i) - C_1 \leq \tau_2^c(t_i) - C_2, \forall t_i \in [\alpha_2, \beta_2]$.

Proof. See the online supplement. $\square$

4.4. Label Joining

When forward and backward labels are symmetric, we can directly sum two cost functions (i.e., $F_{p_2}(t_{p_2})$ and $B_{p_2}(t_{p_2})$ in Figure 6) to obtain the cost of a completed route. However, when labels are asymmetric, transformations are needed before the joining operation. Take the case in Figure 6 as an example. For the backward cost function τ^b , its argument is the SST t_{p_2} at the joining point p_2 , whereas the argument of the forward cost function τ^f is the SST t_0 at depot 0; thus, we need to use function $\phi(t_{p_2})$ to derive t_0 first given the SST at node p_2 is t_{p_2} , that is, $t_0 = \phi(t_{p_2})$. Then we can sum functions $\tau^f(\phi(t_{p_2}))$ and $\tau^b(t_{p_2})$ (as they are both functions of variable t_{p_2}) to obtain the cost of a completed route.

Now suppose we have a forward label L^f and a backward label L^b ending at the same customer i . If i is a pickup customer, let $U'_1 = \{v_d \mid v_d = v_p + n, \forall v_p \in U^f \setminus \{i\}\}$ and $U'_2 = U^b$. If i is a delivery customer, let $U'_1 = \{v_d \mid v_d = v_p + n, \forall v_p \in U^f\}$ and $U'_2 = U^b \setminus \{i\}$. Labels L^f and L^b can be joined together at node i if the following constraints are satisfied:

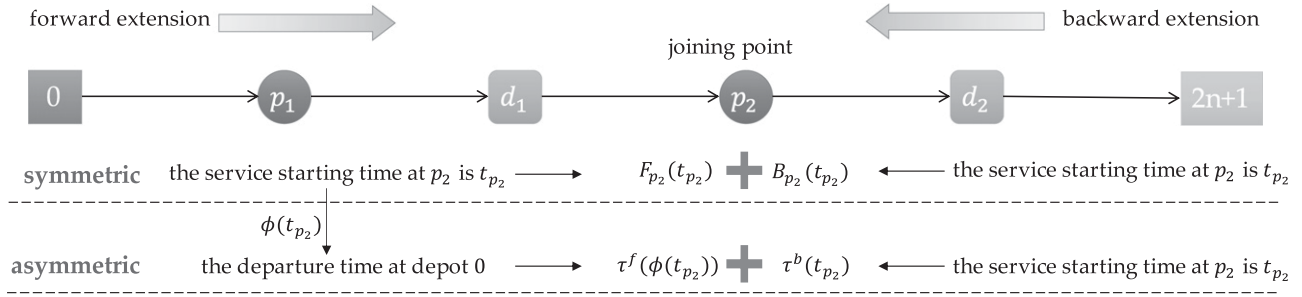
$$\mathbb{P}(L^f) \cap \mathbb{P}(L^b) = \{i\}, \quad (40)$$

$$\max\{\alpha^f, \alpha^b\} \leq \min\{\beta^f, \beta^b\}, \quad (41)$$

$$U'_1 = U'_2. \quad (42)$$

The reduced cost $\bar{c}(r)$ of a completed route r is the minimum of the sum of the forward and backward reduced costs, that is,

$$\bar{c}(r) = \min_{\max\{\alpha^f, \alpha^b\} \leq t_i \leq \min\{\beta^f, \beta^b\}} \{\eta(t_i) + \tau^b(t_i) - \pi_i t_i \mathbb{1}(i \in V_d)\} - C^f - C^b + \mu_i, \quad (43)$$

Figure 6. (Color online) The Symmetric and Asymmetric Label Joining Operations

where $\mathbb{1}(\cdot)$ is an indicator function. Let

$$F(t_i) = \eta(t_i) + \tau^b(t_i) - \pi_i t_i \mathbb{1}(i \in V_d),$$

$$\forall t_i \in [\max\{\alpha^f, \alpha^b\}, \min\{\beta^f, \beta^b\}]. \quad (44)$$

To find the minimum of $F(t_i)$, we only discuss the case where $\max\{\alpha^f, \alpha^b\} < \min\{\beta^f, \beta^b\}$, as $F(t_i)$ is fixed when $\max\{\alpha^f, \alpha^b\} = \min\{\beta^f, \beta^b\}$. When $\max\{\alpha^f, \alpha^b\} < \min\{\beta^f, \beta^b\}$, we must have $\alpha^f < \beta^f$. Note that $\delta^{-1}(t_i) = t_i - \omega$, $\forall t_i \in (\alpha^f, \beta^f]$. To analyze $F(t_i)$, we restate the definition of function $\eta(t_i)$ (see the online supplement) as follows:

$$\eta(t_i) = \tau^f(\phi(t_i)) = \begin{cases} \tau^f(t'_0) & \text{if } t_i = \alpha^f, \\ \tau^f(t_i - \omega) & \text{if } t_i > \alpha^f, \end{cases} \quad (45a)$$

$$(45b)$$

where

$$t'_0 = \arg \min_{t_0 \in [0, \alpha^f - \omega]} \tau^f(t_0). \quad (46)$$

From the definition we know that $\tau^f(t'_0) \leq \tau^f(\alpha^f - \omega)$, so $t_i = \alpha^f$ could be a discontinuity point of function $\eta(t_i)$. The graph of $\tau^f(t_i - \omega)$ can be obtained by shifting the graph of $\tau^f(t_0)$ to the right. In this way, we can get the graph of $\eta(t_i)$ as shown in Figure 7(a). The visualization of $\tau^b(t_i)$ is given in Figure 7(b). Because both $\eta(t_i)$ and $\tau^b(t_i)$ are piecewise linear convex functions, $F(t_i)$ is also a piecewise linear convex function as shown in Figure 7(c). Note that if $\tau^f(t'_0) < \tau^f(\alpha^f - \omega)$, $t_i = \alpha^f$ would also be a discontinuity point of function $F(t_i)$.

By expanding $\eta(t_i)$ and $\tau^b(t_i)$, we get the following proposition.

Proposition 5. When $t_i \in (\alpha^f, \min\{\beta^f, \beta^b\}]$,

$$F(t_i) = \max_{\mathcal{F}_h \in \mathcal{F}} \left\{ \max_{\hat{C}_h \in \mathcal{C}} \{ (u_h + \hat{u}_h - \rho_1 - \pi_i \mathbb{1}(i \in V_d)) t_i + g_h + \hat{g}_h - u_h \omega \} \right\}. \quad (47)$$

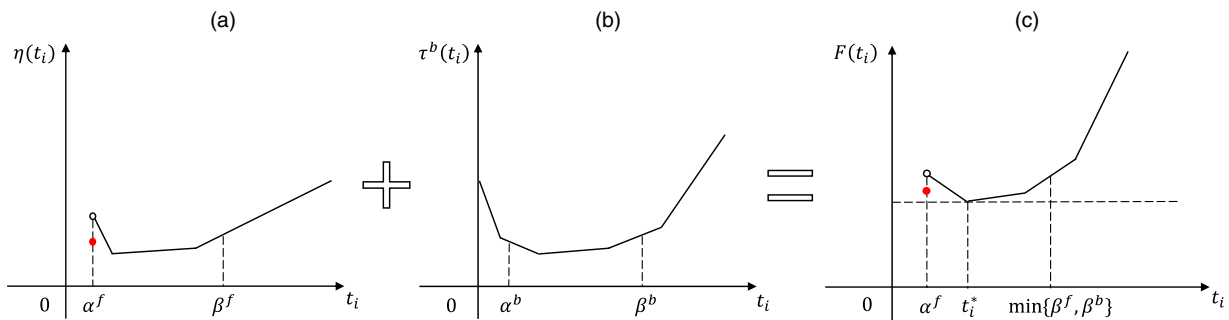
Proof. See the online supplement. $\square$

Up to now, we have known the expression and graph of function $F(t_i)$; thus, we can get its minimum on the interval $[\max\{\alpha^f, \alpha^b\}, \min\{\beta^f, \beta^b\}]$ and also the value of $\bar{c}(r)$.

4.5. Ng-Route Relaxation

Besides the bidirectional search, we also use the *ng*-route relaxation introduced by Baldacci, Mingozzi, and Roberti (2011) to accelerate the labeling algorithm. The *ng*-route concepts relies on the definition of an *ng*-set $\mathcal{N}_i$ for each customer $i \in V_c$. An *ng*-set $\mathcal{N}_i$ contains the closest λ customers to i and node i itself. A cycle $(v_1, v_2, \dots, v_k, v_1)$ is called an *ng*-cycle if $v_1 \in \cap_{i=1, \dots, k} \mathcal{N}_{v_i}$. A route that does not contain any *ng*-cycle is an *ng*-route.

In our labeling algorithm, the *ng*-route relaxation allows each request to be visited more than once under two limitations: (1) the pickup node cannot be revisited before its delivery node is visited, and (2) the

Figure 7. (Color online) Visualizations of Functions $\eta(t_i)$, $\tau^b(t_i)$, and $F(t_i)$ 

precedence constraints associated with the request must be respected. For each customer $i \in V_c$, we define two ng -sets $\mathcal{N}_i^f$ and $\mathcal{N}_i^b$ for the forward and backward extensions, respectively. Distance is used as the criterion to define the proximity of customers. The distance $dist(i, j)$ of two requests is defined as

$$dist(i, j) = \min\{\sigma_{ij}, \sigma_{i,j+n}, \sigma_{i+n,j}, \sigma_{i+n,j+n}\}, \forall i, j \in V_p.$$

Then, for each request $i \in V_p$, the pickup customers of the λ closest requests and i itself will be added to $\mathcal{N}_i^f$. We define $\mathcal{N}_i^f = \mathcal{N}_{i-n}^f, \forall i \in V_d$ and $\mathcal{N}_i^b = \{j+n \mid j \in \mathcal{N}_i^f\}, \forall i \in V_c$.

For a forward label L^f and its associated route $P(L^f) = (v_0, v_1, \dots, v_m)$, let $ng(L^f)$ be the subset of requests such that an ng -cycle is prohibited when extending L^f . Then $ng(L^f)$ can be written as

$$ng(L^f) = \left\{ i \in V_p \mid \exists k \in \{1, \dots, m\} \text{ such that } n+i = v_k \text{ and } i \in \bigcap_{i'=k}^m \mathcal{N}_{v_{i'}}^f \right\}.$$

For a backward label L^b and its associated route $P(L^b) = (v_0, v_1, \dots, v_m)$, $ng(L^b)$ is defined as

$$ng(L^b) = \left\{ i \in V_d \mid \exists k \in \{1, \dots, m\} \text{ such that } i-n = v_k \text{ and } i \in \bigcap_{i'=k}^m \mathcal{N}_{v_{i'}}^b \right\}.$$

When using the ng -route relaxation in the pricing problem, for the forward extension, we need to update Condition (11) and set Γ^f in (14) as

$$j \in V_p \wedge (\varphi_j \oplus \Psi(U^f)) \wedge (j \notin U^f) \text{ and}$$

$$\Gamma^f = V_c \cup \{2n+1\} \setminus ng(L^f) \setminus \mathcal{A}^f \setminus \mathcal{B}^f,$$

respectively. Similarly, for the backward extension, we update Condition (28) and set $\Gamma^{b'}$ as

$$j \in V_d \wedge (\varphi_j \oplus \Psi(U^b)) \wedge (j \notin U^b) \text{ and}$$

$$\Gamma^{b'} = V_c \cup \{0\} \setminus ng(L^{b'}) \setminus \mathcal{A}^b \setminus \mathcal{B}^b,$$

respectively. Meanwhile, the joining Condition (40) ensuring elementary routes is not needed.

5. Branching Strategy

We use the best-first strategy to explore the B&B tree, that is, priority is given to an unexplored tree node whose father node has the smallest lower bound. In our B&P algorithm, we use the following hierarchical branching scheme. First, we branch on the number of used vehicles. Let $(\tilde{\theta}_r)(r \in \mathcal{R})$ be an optimal solution of the LRP and $\tilde{m} = \sum_{r \in \mathcal{R}} \tilde{\theta}_r$ be the number of used vehicles in the optimal solution. If $\tilde{m}$ is fractional, we generate two children nodes by forcing $\sum_{r \in \mathcal{R}} \theta_r \leq \lfloor \tilde{m} \rfloor$ and $\sum_{r \in \mathcal{R}} \theta_r \geq \lceil \tilde{m} \rceil$, respectively. If $\tilde{m}$ is integer, we

branch on a fractional arc. Let the total flow through arc $(i, j) \in A$ in the optimal solution be $\tilde{x}_{ij} = \sum_{r \in \mathcal{R}} \tilde{\gamma}_{ijr} \tilde{\theta}_r$, where $\tilde{\gamma}_{ijr}$ denotes whether arc (i, j) is traversed by route r . If there exist multiple arcs with fractional values of $\tilde{x}_{ij}$, we then branch on arc (i^*, j^*) whose value of $\tilde{x}_{i^*j^*}$ is the closest to 0.5. Two children nodes are generated by forbidding and enforcing vehicles to travel through arc (i^*, j^*) in the labeling algorithm, respectively. We can implement the former case by deleting arc (i^*, j^*) . For the latter case, we delete arcs $(i^*, l), l \neq j^*$ and $(l, j^*), l \neq i^*$.

6. Computational Experiments

In this section, we introduce the instances and evaluate the B&P algorithm and the new objective function. Our algorithm was coded in Java programming language using ILOG CPLEX 12.10 as the solver. All parameters in the solver were set to their default values. The RMPs in the CG framework were solved by the simplex algorithm. All experiments were run on a laptop with a 2.5 GHz Intel(R) Core(TM) i5-7300HQ processor and 24 GB of memory under the macOS system using a single thread. The time limit on each execution of the B&P algorithm was set to 7,200 seconds, and all the computing times in this section are reported in seconds.

6.1. Instances

We generate PDPTW-IC instances based on the PDPTW instances proposed by Li and Lim (2008), which originate from the Solomon (1987) VRPTW instances by randomly pairing up customers in the same route. Like Solomon's instance classification, the PDPTW instances also have six classes: LC1, LC2, LR1, LR2, LRC1, and LRC2. Customers are clustered in LC instances and randomly distributed in LR instances. A subset of customers are clustered, and the others are randomly distributed in LRC instances. The LC1, LR1, and LRC1 instances have a short time horizon, while the LC2, LR2, and LRC2 instances have a long time horizon. All instances have more than 50 requests, and some customers are dummy nodes for coupling purposes.

We generate three instance sets with different time horizons, where *Set Short* and *Set Normal* instances are adapted from the LRC1 and LC1 instances, respectively. These two instance sets are used to simulate short-distance and long-distance urban distributions, respectively. In particular, we set four hours for *Set Short* and one day for *Set Normal*. The *Set Large* instances are modified from the LC2 instances, which have a long time horizon—around three days. To simulate long-distance trunk transportation, the travel distances between nodes in *Set Large* instances are tripled. In each instance set, the number of customers takes values from the set $\{10, 20, 30, 40, 50, 60, 80\}$. Based on the transportation cost of refrigerated

vehicles provided by the 3PL company, we set $\rho_0 = 1.2$ and $\rho_1 = 0.4$.

Each PDPTW-IC instance includes information on customer, vehicle, and commodity. For a customer, its location, demand, and time window are known. For a vehicle, its travel speed and capacity are known. We set node coordinates, time windows, service times, and vehicle speeds the same as those in Li and Lim (2008). We also keep the same pickup-delivery pairs. For each of their instances, we rank the pairs based on the order the pickup customer appears. Then we generate instances with $n = 10, \dots, 80$ customers by selecting the first $n/2$ pairs. We enlarge customer demands and vehicle capacities 10 times to reflect real-world statistical data. For each commodity, its perishability or stability, price, and (in)compatibility are known. We mainly consider four types of commodities: fruit and vegetable, poultry and meat, medical supply, and manufactured food. We assume fruit and vegetable are perishable, whereas the others are stable under cold chain conditions. In each instance, the proportion of perishable commodities is around 30%. When collecting the data, we found that the *harvest period* was quite long, especially for the imported fruits and vegetables, which was normally greater than the *fresh period*. As a result, most perishable products began to decay visibly before time e_0 . Thus, we set $\zeta'_\varphi = 0$, $\forall \varphi \in \overline{\mathcal{P}}$ in the following experiments. We also assume that a perishable product at time ζ'' can only be sold at a half price of that at time ζ' . The price differences and time spans of visible changes for all perishable products are detailed in the online supplement, which are based on the local market data in Wuhan, China. Based on some preliminary tests, we set the neighborhood size λ of the *ng*-route relaxation to 15. The details of instances and solutions are also available at <https://github.com/dengfaheng/PDPTW-IC-INST>.

6.2. Evaluation of Label Extension Method

As suggested before, the forward extension is quite different from the backward extension in our labeling algorithm. Thus, our first attempt is to evaluate the efficiency of different extension methods, including only forward extension (FE), only backward extension (BE), and bidirectional extension (BidE). The FE (BE) can be implemented by setting $\Theta^f = l_0 + 1, \Theta^b = e_0 - 1$ ($\Theta^f = e_0 - 1, \Theta^b = l_0 + 1$), and the BidE can be achieved by setting $\Theta^f = \Theta^b = l_0/2$. We conduct experiments by selecting instances of different sizes from each set. Results are reported in Table 3, where columns n and UB denote the number of customers and the optimal value of each instance. Columns $F.T$, $B.T$, and $Bid.T$ represent the computing times of using FE, BE, and BidE, respectively.

From Table 3, we observe that all the selected instances in Set Short are solved to optimality and the time differences among the three methods are minor. However, for Set Normal, although the three methods can provide optimal solutions for all the instances, the total computing time of BidE is shorter than those of FE and BE. For instances in Set Large, BE cannot provide a feasible solution for the 80-customer instance within the 7,200-second time limit. Although FE can produce optimal solutions for all the selected instances, its computing time is much higher than that of BidE, that is, 1,673.29 seconds against 388.93 seconds. From the results, we can conclude that the bidirectional label extension method is more efficient in solving PDPTW-IC instances, particularly for large-size instances with a long time horizon. Therefore, in the following sections, we use BidE to perform our experiments.

6.3. Evaluation of the B&P Algorithm

In this section, we first report the results of our B&P algorithm for all the instances, which are summarized

Table 3. Performance Comparisons Among the Three Label Extension Methods

n	Set short				Set normal				Set large			
	UB	F.T	B.T	Bid.T	UB	F.T	B.T	Bid.T	UB	F.T	B.T	Bid.T
10	411.66	0.19	0.01	0.10	994.47	0.01	0.01	0.01	4,330.39	0.01	0.01	0.01
10	420.29	0.03	0.01	0.02	920.60	0.02	0.01	0.01	2,512.64	0.02	0.01	0.01
20	725.58	0.05	0.01	0.01	1,271.89	0.01	0.01	0.01	4,681.68	0.01	0.01	0.01
20	723.05	0.12	0.04	0.04	1,405.97	1.40	0.93	0.88	5,174.13	0.13	0.10	0.27
30	1,188.09	0.12	0.12	0.14	1,792.24	0.02	0.05	0.02	6,001.87	0.08	0.43	0.08
30	1,061.70	0.05	0.06	0.04	1,858.94	0.14	0.16	0.06	5,819.13	1.90	1.04	0.64
40	1,878.65	0.74	0.04	0.03	2,542.21	0.02	0.03	0.02	7,724.19	0.23	0.84	0.30
40	1,419.37	0.21	0.16	0.06	2,544.52	4.02	15.62	6.62	8,025.10	2.40	3.33	3.40
50	2,237.43	0.39	0.38	0.34	3,324.36	0.02	0.04	0.02	10,909.69	0.08	0.98	0.17
50	1,753.96	0.10	0.06	0.05	3,029.92	0.29	1.49	0.17	10,625.60	10.72	6.46	5.58
60	2,740.03	0.18	0.25	0.18	4,699.65	0.09	0.51	0.10	12,549.44	0.17	15.06	2.90
60	2,346.77	0.14	0.11	0.07	3,932.81	6.21	18.15	1.84	12,415.15	334.21	94.51	31.14
80	3,648.41	0.14	0.26	0.13	6,197.25	7.92	185.31	2.50	15,554.95	1,323.33	N/A	344.42
Sum	—	2.46	1.51	1.21	—	20.17	222.32	12.26	—	1,673.29	122.78	388.93

Note. N/A, no feasible solution is provided within the 7,200-second time limit.

Table 4. A Summary of Results Generated by Our B&P Algorithm for All the Instances

<i>n</i>	Set short				Set normal				Set large			
	Total	Solved	Time	Node	Total	Solved	Time	Node	Total	Solved	Time	Node
10	8	8	0.05	1.25	9	9	0.01	1.00	8	8	0.02	1.00
20	8	8	0.20	2.25	9	9	0.62	6.56	8	8	0.15	1.00
30	8	8	0.69	9.75	9	9	1.65	1.00	8	8	36.12	1.25
40	8	8	7.92	173.25	9	9	98.79	18.78	8	8	108.42	2.00
50	8	8	749.37	17,130.00	9	9	30.97	1.67	8	8	332.71	1.50
60	8	8	664.43	2,832.00	9	8	180.90	24.25	8	7	450.26	1.57
80	8	8	1,488.31	1,186.75	9	5	126.38	1.00	8	2	2,930.68	1.00
Sum	56	56	2,910.98	21,335.25	63	58	439.31	54.25	56	49	3,858.36	9.32

Note. Results in columns *Time* and *Node* are the average results of instances that are solved to optimality.

in Table 4; the detailed results are presented in the online supplement. In Table 4, column *Total* denotes the total number of instances. Column *Solved* reports the number of instances that are solved to optimality. For these optimally solved instances, the average computing time and the average number of explored nodes in the B&B tree are reported in columns *Time* and *Node*, respectively.

For Set Short, all the 56 instances are solved to optimality by B&P. This is because instances in this set have relatively narrow time windows and thus are the most tractable in comparison with instances in other sets. Moreover, we observe that the B&B process has explored a large number of nodes for instances in this set. For Set Normal, 58 out of 63 instances are solved to optimality with much fewer explored nodes. The instances in Set Large are the most difficult because of their wider time windows and longer time horizons, and thus 7 out of 56 instances cannot be optimally solved. Overall, the B&P algorithm can produce optimal solutions for 163 out of 175 instances within an acceptable time frame. Tables in the online supplement further show that for most instances, the lower bound at the root node is very tight and B&P can find the optimal solution by only exploring the root node, proving the effectiveness of our set partitioning model and B&P algorithm.

Table 5 reports the performance of dominance rules at the root node of the B&B tree, where columns *Label*

and *Dominated* are the average numbers of generated and dominated labels, respectively. Column *Ratio* (%) provides the percentage of dominated labels, computed as $Ratio\ (%) = Dominated/Label \times 100$. We observe that 52.51% of labels can be dominated for Set Short instances, and the ratio increases to 70.47% for Set Large instances. For these dominated labels, we can directly discard them and also not need to consider their extensions, reducing the overall computing time of our B&P algorithm.

6.4. Analyses of Cost Components and Objective Functions

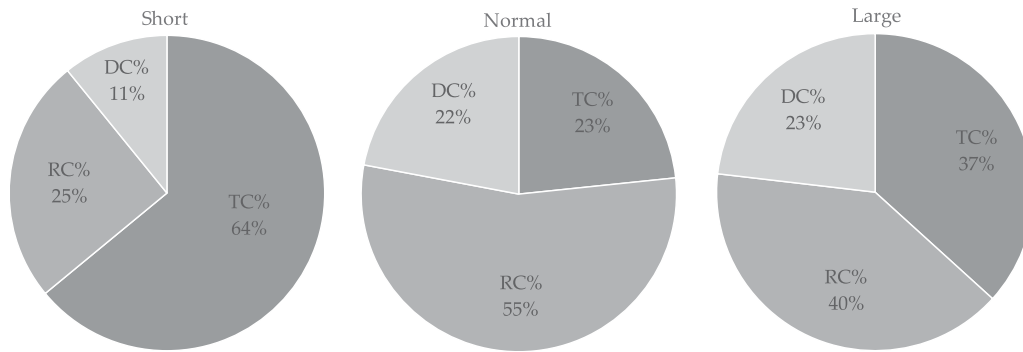
In this section, we analyze some important features of PDPTW-IC. For all the optimally solved instances in each set, we calculate the average proportions of travel cost (*TC*%), refrigeration cost (*RC*%), and quality decay cost (*DC*%) in the total cost, which are reported in Figure 8. From the results, we get some interesting observations:

1. In short-distance urban distribution (Set Short), a majority (64%) of the total cost comes from the travel cost, and only 11% is the quality decay cost. This observation makes sense because instances in this set have a short time horizon, and thus products are picked up from and delivered to customers quickly. Consequently, perishable products can keep a high level of freshness when delivered, leading to a low-quality decay cost.

Table 5. Performance of Dominance Rules at the Root Node of the B&B Tree

<i>N</i>	Set short			Set normal			Set large		
	Label	Dominated	Ratio (%)	Label	Dominated	Ratio (%)	Label	Dominated	Ratio (%)
10	977	357	36.53	203	72	35.21	820	394	48.08
20	3,685	1,755	47.62	3,696	1,636	44.27	8,733	5,719	65.49
30	8,002	4,104	51.28	26,726	16,677	62.40	136,613	105,296	77.08
40	10,457	5,768	55.16	92,656	59,317	64.02	157,579	122,506	77.74
50	9,720	5,513	56.71	83,562	52,655	63.01	611,115	480,879	78.69
60	24,419	14,442	59.14	89,326	52,959	59.29	934,360	726,194	77.72
80	132,254	80,863	61.14	336,025	168,042	50.01	2,036,989	1,395,653	68.52
Avg.	27,073	16,114	52.51	90,313	50,194	54.03	555,173	405,234	70.47

Note. Avg., average.

Figure 8. The Average TC%, RC%, and DC% of All Optimally Solved Instances in Each Set

2. In long-distance urban distribution (Set Normal) and trunk transportation (Set Large), the refrigeration cost becomes the most crucial part, and the quality decay cost also doubles. As mentioned earlier, both traveling and waiting contribute to the refrigeration cost. For Set Normal instances, the high refrigeration cost mainly comes from the long waiting times at customers. For Set Large instances, because we have tripled the travel distances between nodes, the waiting times at customers are relatively shorter; thus, the proportion of the refrigeration cost is not as high as that in Set Normal instances. As waiting at customers or on the roads (due to traffic congestions) is quite common in real-world transportation activities, it is vital to take the refrigeration cost into consideration when making scheduling decisions for refrigerated vehicles. In addition, as the time horizon becomes longer, the quality decay cost also increases because of the limited life spans of perishable products.

To the best of our knowledge, most 3PL companies in China take travel costs as the first or even the only consideration when planning logistics activities, which may increase the refrigeration and quality decay costs, leading to a high total cost. To validate the importance of a comprehensive objective function, we compare results under different objectives. For this purpose, we solve all the optimally solved instances again but with

the objective of minimizing the travel cost, which can be implemented by setting $\rho_1 = 0$ and $\pi_i = 0, \forall i \in V_d$. We then fix the scheduling decisions and calculate the corresponding refrigeration cost, quality decay cost, and total cost. Detailed results under this objective function can be found in the online supplement. Comparisons of the total cost under different objective functions are reported in Table 6. Column *TRDC* gives the average result under our proposed objective function, and column *TCO* presents the average result with the objective of minimizing the travel cost. Columns *Rd (%)* and *#Rd* provide the percentage and quantity differences, respectively. They are calculated as $Rd (\%) = (TRDC - TCO)/TCO \times 100$ and $\#Rd = TRDC - TCO$. Thus, a negative value of *Rd (%)* or *#Rd* means that our proposed objective function has a lower total cost.

From Table 6, we observe that our comprehensive objective function can help reduce the total cost for all the instance sets. In particular, as the time horizon becomes longer, cost reduction becomes more significant, from 5.41% for Set Short to 16.36% for Set Large. For 3PL companies, the CCT activity is performed several times per week. Hence, the accumulated cost reduction over an entire year could be significant. To conclude, besides the travel cost, refrigeration and quality decay costs are also important factors that

Table 6. Comparisons of the Total Cost Under Different Objective Functions

N	Set short				Set normal				Set large			
	TRDC	TCO	Rd (%)	#Rd	TRDC	TCO	Rd (%)	#Rd	TRDC	TCO	Rd (%)	#Rd
10	390.78	422.54	-7.35	-31.76	926.12	1,135.93	-19.06	-209.81	2,768.07	3,101.32	-10.71	-333.25
20	658.56	701.64	-5.74	-43.08	1,287.17	1,479.01	-12.86	-191.84	4,194.32	5,186.47	-19.15	-992.15
30	964.72	1,027.16	-6.08	-62.44	1,737.21	2,009.55	-13.35	-272.34	4,975.83	6,338.93	-20.77	-1,363.10
40	1,416.32	1,488.88	-4.69	-72.56	2,430.93	2,836.70	-14.03	-405.77	6,778.71	8,247.54	-17.74	-1,468.83
50	1,741.68	1,828.88	-4.54	-87.20	3,055.27	3,589.80	-14.68	-534.54	8,393.24	9,977.40	-15.84	-1,584.16
60	2,127.90	2,239.58	-4.76	-111.68	4,122.18	4,822.36	-14.29	-700.17	10,813.65	13,115.14	-17.66	-2,301.48
80	2,722.79	2,863.87	-4.73	-141.07	5,768.50	6,617.01	-12.78	-848.51	15,554.95	17,802.88	-12.63	-2,247.93
Avg.	1,431.82	1,510.36	-5.41	-78.54	2,761.06	3,212.91	-14.43	-451.85	7,639.83	9,109.95	-16.36	-1,470.13

Note. Avg., average.

should be considered when planning CCT activities, especially long-distance transportation activities.

7. Conclusions

This study introduces a new cold chain transportation problem extended from the classic PDPTW, where multiple commodities with incompatibility constraints are considered. For the new variant, besides travel costs, refrigeration costs of vehicles and quality decay costs of perishable products are also explicitly incorporated into the objective function. To minimize the total cost, we focus on determining the optimal pickup and delivery routes and the vehicle departure times from the depot. We develop a set partitioning model for the problem, which is solved by a tailored B&P algorithm. To tackle the asymmetry issue arising from the pricing problem of the B&P framework, we develop a novel asymmetric bidirectional labeling algorithm. Benchmark instances based on real-world statistical data and known instance sets in the literature are first generated for this problem. Extensive numerical results show that the bidirectional labeling algorithm has better performance than the monodirectional labeling algorithm and that our B&P algorithm can solve most instances to optimality in an acceptable time frame. In addition, our proposed objective function can help reduce the total cost, compared with the widely adopted objective function minimizing only the travel cost.

Future research could be conducted from two aspects: (1) In this paper, we assume that the travel times between nodes are constant, whereas they are closely related to traffic conditions in real-world applications. Thus, considering time-dependent travel times can result in a more practical problem. (2) More powerful algorithms can be developed to solve large-size instances with a long time horizon more efficiently. For example, we can consider setting dynamic half-way points (i.e., values of parameters Θ^f and Θ^b) as in Tilk et al. (2017) to further improve the efficiency of the bidirectional labeling algorithm. We can also develop heuristic or metaheuristic algorithms.

Acknowledgments

The authors are thankful to the associate editor and two anonymous referees for their helpful comments.

References

- Amorim P, Almada-Lobo B (2014) The impact of food perishability issues in the vehicle routing problem. *Comput. Indust. Engrg.* 67: 223–233.
- Baldacci R, Bartolini E, Mingozzi A (2011) An exact algorithm for the pickup and delivery problem with time windows. *Oper. Res.* 59(2):414–426.
- Baldacci R, Mingozzi A, Roberti R (2011) New route relaxation and pricing strategies for the vehicle routing problem. *Oper. Res.* 59(5):1269–1283.
- Battarra M, Monaci M, Vigo D (2009) An adaptive guidance approach for the heuristic solution of a minimum multiple trip vehicle routing problem. *Comput. Oper. Res.* 36(11):3041–3050.
- Bettinelli A, Cacchiani V, Crainic TG, Vigo D (2019) A branch-and-cut-and-price algorithm for the multi-trip separate pickup and delivery problem with time windows at customers and facilities. *Eur. J. Oper. Res.* 279(3):824–839.
- Ceselli A, Righini G, Salani M (2009) A column generation algorithm for a rich vehicle-routing problem. *Transportation Sci.* 43(1):56–69.
- Chai X (2016) Barcode solutions help companies reduce food distribution losses. *Logist. Material Handling* 22(9):36–37.
- Chen HK, Hsueh CF, Chang MS (2009) Production scheduling and vehicle routing with time windows for perishable food products. *Comput. Oper. Res.* 36(7):2311–2319.
- Cherkesly M, Desaulniers G, Laporte G (2015) Branch-price-and-cut algorithms for the pickup and delivery problem with time windows and last-in-first-out loading. *Transportation Sci.* 49(4): 752–766.
- Cnwest (2015) Why is fresh online shopping “not fresh”? Cold chain distribution becomes a “pain point.” Accessed April 18, 2021, <https://society.huanqiu.com/article/9CaKrnjMvlp>.
- Cordeau J-F (2006) A branch-and-cut algorithm for the dial-a-ride problem. *Oper. Res.* 54(3):573–586.
- Cordeau J-F, Laporte G, Ropke S (2008) Recent models and algorithms for one-to-one pickup and delivery problems. *The Vehicle Routing Problem: Latest Advances and New Challenges* (Springer, New York), 327–357.
- Costa L, Contardo C, Desaulniers G (2019) Exact branch-price-and-cut algorithms for vehicle routing. *Transportation Sci.* 53(4):946–985.
- Dabia S, Ropke S, Van Woensel T, De Kok T (2013) Branch and price for the time-dependent vehicle routing problem with time windows. *Transportation Sci.* 47(3):380–396.
- Desaulniers G, Desrosiers J, Solomon MM (2005) *Column Generation*, vol. 5 (Springer, New York).
- Factorovich P, Méndez-Díaz I, Zabala P (2020) Pickup and delivery problem with incompatibility constraints. *Comput. Oper. Res.* 113:104805.
- Feillet D, Dejax P, Gendreau M, Gueguen C (2004) An exact algorithm for the elementary shortest path problem with resource constraints: Application to some vehicle routing problems. *Networks* 44(3):216–229.
- Furtado MGS, Munari P, Morabito R (2017) Pickup and delivery problem with time windows: A new compact two-index formulation. *Oper. Res. Lett.* 45(4):334–341.
- Gendreau M, Manerba D, Mansini R (2016) The multi-vehicle traveling purchaser problem with pairwise incompatibility constraints and unitary demands: A branch-and-price approach. *Eur. J. Oper. Res.* 248(1):59–71.
- Goeke D (2019) Granular tabu search for the pickup and delivery problem with time windows and electric vehicles. *Eur. J. Oper. Res.* 278(3):821–836.
- Goksal FP, Karaoglan I, Altıparmak F (2013) A hybrid discrete particle swarm optimization for vehicle routing problem with simultaneous pickup and delivery. *Comput. Indust. Engrg.* 65(1): 39–53.
- Gschwind T, Irnich S, Rothenbächer A-K, Tilk C (2018) Bidirectional labeling in column-generation algorithms for pickup-and-delivery problems. *Eur. J. Oper. Res.* 266(2):521–530.
- Hsu C-I, Hung S-F, Li H-C (2007) Vehicle routing problem with time-windows for perishable food delivery. *J. Food Engrg.* 80(2): 465–475.
- Li H, Lim A (2008) A metaheuristic for the pickup and delivery problem with time windows. *Internat. J. Artificial Intelligence Tools* 12(02):173–186.
- Manerba D, Mansini R (2015) A branch-and-cut algorithm for the multi-vehicle traveling purchaser problem with pairwise incompatibility constraints. *Networks* 65:139–154.

- Osvald A, Stirn LZ (2008) A vehicle routing algorithm for the distribution of fresh vegetables and similar perishable food. *J. Food Engrg.* 85(2):285–295.
- Parragh SN, Doerner KF, Hartl RF (2008a) A survey on pickup and delivery problems Part I: Transportation between customers and depot. *J. Für Betriebswirtschaft* 58(1):21–51.
- Parragh SN, Doerner KF, Hartl RF (2008b) A survey on pickup and delivery problems Part II: Transportation between pickup and delivery locations. *J. Für Betriebswirtschaft* 58(2): 81–117.
- Pessoa A, Sadykov R, Uchoa E, Vanderbeck F (2020) A generic exact solver for vehicle routing and related problems. *Math. Programming* 183(1):483–523.
- Pisinger D, Ropke S (2007) A general heuristic for vehicle routing problems. *Comput. Oper. Res.* 34(8):2403–2435.
- Qi C, Hu L (2020) Optimization of vehicle routing problem for emergency cold chain logistics based on minimum loss. *Physical Comm.*, 1–7.
- Qin H, Su X, Ren T, Luo Z (2021) A review on the electric vehicle routing problems: Variants and algorithms. *Frontiers Engrg. Management* 8(3):370–389.
- Righini G, Salani M (2006) Symmetry helps: Bounded bi-directional dynamic programming for the elementary shortest path problem with resource constraints. *Discrete Optim.* 3(3):255–273.
- Ropke S, Cordeau JF (2009) Branch and cut and price for the pickup and delivery problem with time windows. *Transportation Sci.* 43(3):267–286.
- Ropke S, Cordeau J-F, Laporte G (2007) Models and branch-and-cut algorithms for pickup and delivery problems with time windows. *Networks* 49(4):258–272.
- Shanghai Municipal Quality and Technical Supervision Bureau (2007) Technology and management specification of cold chain logistics (DB 31/T 388-2007). Accessed April 18, 2021, <http://www.bjqtwl.com/html/shejiguifan/2014/0417/76.html>.
- Solomon MM (1987) Algorithms for the vehicle routing and scheduling problems with time window constraints. *Oper. Res.* 35(2): 254–265.
- Song BD, Ko YD (2016) A vehicle routing problem of both refrigerated- and general-type vehicles for perishable food products delivery. *J. Food Engrg.* 169:61–71.
- Stellingwerf HM, Groeneveld LHC, Laporte G, Kanellopoulos, Bloemhof JM, Behdani B (2021) The quality-driven vehicle routing problem: Model and application to a case of cooperative logistics. *Internat. J. Production Econom.* 231:107849.
- Tilk C, Rothenbächer A-K, Gschwind T, Irnich S (2017) Asymmetry matters: Dynamic half-way points in bidirectional labeling for solving shortest path problems with resource constraints faster. *Eur. J. Oper. Res.* 261(2):530–539.
- Veenstra M, Cherkesly M, Desaulniers G, Laporte G (2017) The pickup and delivery problem with time windows and handling operations. *Comput. Oper. Res.* 77:127–140.
- Wang C, Mu D, Zhao F, Sutherland JW (2015) A parallel simulated annealing method for the vehicle routing problem with simultaneous pickup–delivery and time windows. *Comput. Indust. Engrg.* 83:111–122.

Copyright 2023, by INFORMS, all rights reserved. Copyright of Transportation Science is the property of INFORMS: Institute for Operations Research and its content may not be copied or emailed to multiple sites or posted to a listserv without the copyright holder's express written permission. However, users may print, download, or email articles for individual use.